TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Solving Problems Using String Functions

Text in C is an array of characters with a sentinel at the end: the null character. Everything in this experiment — length, copying, joining, comparing — is the library walking that array, and every classic string bug is forgetting the sentinel exists.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 7 · LEVEL 2 TERM 1

1. Aim

To use the standard string library (<string.h>) — strlen, strcpy, strcat, strcmp — and hand-written character loops to solve text problems: measuring, reversing, counting and testing strings.

2. Concepts you need

- **What a string is:** a char array whose valid characters are followed by **'\0'** (the null character, value 0). "Textile" occupies 8 elements, not 7. A char array without a '\0' is not a string and no library function may be trusted with it.
- **Reading text:** scanf("%s", name) reads one word and stops at whitespace — and needs no & (an array's name already acts as its address). For whole lines including spaces, use fgets, which also limits the length — the safe habit. The old gets() is removed from modern C for good reason: it cannot limit input and overflows silently.
- **The library (needs #include <string.h>):**

FUNCTION	JOB	CONCRETE VALUE
strlen(s)	Length, not counting '\0'	strlen("Textile") = 7
strcpy(dest, src)	Copies src (with its '\0') into dest; dest must be big enough	strcpy(d, "Yarn") leaves d holding "Yarn"
strcat(dest, src)	Appends src to the end of dest; dest must have room for both plus '\0'	s = "Spin"; strcat(s, "ning") → "Spinning"
strcmp(a, b)	Compares letter by letter: 0 if equal, negative if a comes first alphabetically, positive if b does	strcmp("cat", "cat") = 0; strcmp("apple", "apply") is negative, because 'e' < 'y' at the first differing position

- **strrev is not standard C** — some compilers offer it, the standard does not. Reversing is therefore written by hand in this experiment, as it must be in portable code.
- **Characters are numbers:** a char holds a code (ASCII: 'A' = 65, 'a' = 97, '0' = 48), which is why comparisons and case tests work arithmetically.

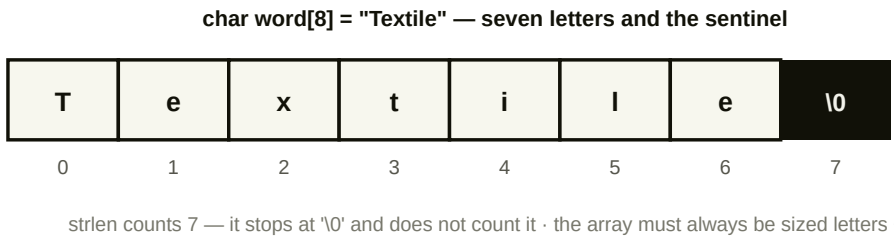


Figure 1 — A string in memory. Every string function in this experiment works by walking from index 0 until it meets the black box.

3. Lab procedure

1. Predict all outputs before running; strings punish guesswork — trace character positions on paper, using Figure 1 as the model.
2. Deliberately declare char small[4] and attempt strcpy(small, "Textile") in a throwaway copy; observe (compiler warning or corrupted behaviour) and record why a destination's size is a correctness condition, not a courtesy.
3. Read a two-word name with scanf("%s") and print what was actually stored — the evidence for the whitespace rule.

4. Worked example — length and hand-written reversal

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char str[40] = "Textile Engineering";
    int i, len;
    len = strlen(str);
    printf("String: %s\n", str);
    printf("Length = %d\n", len);
    printf("Reversed: ");
    for (i = len - 1; i >= 0; i--)
        printf("%c", str[i]);
    printf("\n");
```

```
    return 0;
}
```

Output:

```
String: Textile Engineering
Length = 19
Reversed: gnireenignE elitxeT
```

Hand check: "Textile" (7) + one space (1) + "Engineering" (11) = **19** characters ✓. The reversal loop prints indices 18 down to 0: the last letter 'g' first, the first letter 'T' last, the space travelling to its mirrored position — "gnireenignE elitxeT" ✓. The string itself is unchanged; reversal here is a printing order, not a rearrangement.

5. Practice tasks

1. Write a program that counts the vowels in the string "Programming" (test both cases). Give the output and the per-letter verdicts.
2. Write a program that tests whether "madam" is a palindrome (reads the same backwards). Give the output and the comparisons the loop makes.
3. A word of exactly 10 letters is to be stored and later have 5 more letters appended. State the smallest safe array size, naming everything that must fit.

Answer key

1.
```
#include <stdio.h>

int main(void)
{
    char s[30] = "Programming";
    int i, count = 0;
    char c;
    for (i = 0; s[i] != '\0'; i++)
    {
        c = s[i];
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
            c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U')
            count++;
    }
    printf("Vowels = %d\n", count);
    return 0;
}
```

Verdicts per letter: P no, r no, **o yes (1)**, g no, r no, **a yes (2)**, m no, m no, **i yes (3)**, n no, g no. Output: **Vowels = 3**.

2.
```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[30] = "madam";
    int i, len, flag = 1;
    len = strlen(s);
    for (i = 0; i < len / 2; i++)
    {
        if (s[i] != s[len - 1 - i])
        {
            flag = 0;
            break;
        }
    }
    if (flag)
        printf("%s is a palindrome\n", s);
    else
        printf("%s is not a palindrome\n", s);
    return 0;
}
```

Comparisons: i = 0: s[0] 'm' vs s[4] 'm' ✓; i = 1: s[1] 'a' vs s[3] 'a' ✓; loop ends (only len/2 = 2 comparisons — the middle letter needs no partner). Output: **madam is a palindrome**.

3. The final text is 10 + 5 = 15 letters, and the '\0' must also fit: smallest safe size = **16**. Sizing for the first value alone (11) guarantees the strcat overflows — buffer sizes are chosen for the largest content the array will ever hold.
